

Productive Failure Report — Naive Baseline

Deliverable 3. The failure taxonomy of the naive baseline, and why these failures are **productive**: each maps to a measured D1 assumption and to the capability that removes it (R1–R8). Run: 2026-08-08, first real execution of this workload. All numbers are from `summary.json` / `baseline_results.csv` (committed alongside this report).

1. The falsification headline

A common intuition says: *retrieval quality is the problem — if the right text can be found, memory works*. This baseline **falsifies** that premise on this project's workload:

- `recall@5` = **1.00** — the right text is found in every relevance case.
- ...yet the system is wrong where it matters: contradiction failure rate = **0.33**, sensitive-leak rate (injection) = **1.00**, cold-start false-positive rate = **0.50**, task-level pass rate = **0.67**.

The failure is not in *finding* the memory. It is in **selecting the current truth (R4+supersession), keeping it within budget (R5), governing what enters the store (R1/R3), and knowing when there is nothing relevant (R5 fallback)**. Raw recall is necessary but insufficient — exactly the claim D1 made a priori and this run now measures.

2. Failure taxonomy (each failure is a design requirement in disguise)

#	Failure class	Extent (measured)	Typical case	Capability that removes it
F1	Stale fact outranks current fact	1 of 3 contradiction cases (top1 = the superseded "flexible deadline")	c1-direct-contradiction	Supersession + multi-signal ranking (R4)
F2	Sensitive content injected into context	1.0 retrieval, 1.0 injection — <i>every</i> sensitive case, incl. a DB password surfaced by a hosting query	c5-sensitive-leak-check	Admission at ingest + guardrail at retrieval (R1, R3)
F3	Cold-start false positive	1 of 2 no-relevant cases returned a confident spurious match	c6-no-relevant-memory-exists (taco lunch spot for a testing-framework query)	Confidence floor / "don't know" fallback (R5)
F4	Task-level wrong action	1 of 3 contradiction turns would produce the wrong downstream decision from injected context	c1-direct-contradiction	Contradiction + correction/deletion semantics (R6)

#	Failure class	Extent (measured)	Typical case	Capability that removes it
F5	Cross-user wording collision	c4-b: near-identical wording across users scores high; only a hard user_id filter prevents literal leakage	c4-cross-user-similar-wording-b	Index-level + storage-level isolation, not app-layer only (R3) — Week 2 research red flag confirmed structurally present
F6	Full-recall illusion	recall@5=1.0 coexists with all the above	whole workload	Multi-metric evaluation harness (R8)

3. What the numbers say about the D1 chain

- **precision@1 = 0.857, recall@5 = 1.0** → **retrieval is sound but not sufficient**. The chain's Stage 2 (RAG with naive ranking) is where a real system with real users would *systematically* betray them — exactly the "plausible but wrong" failure.
- **No false cold-start miss** (c6-cold-start-new-user behaved correctly), but the non-empty no-relevant store case failed on a false positive: a store without admission + similarity threshold fabricates "memories" where none exist. This is D1's "cold start" edge case made mechanical.
- **Storage growth**: final_storage_count=2, storage proxy 112 bytes (after last case). Raw growth is undramatic on this workload but structurally unbounded; C8 concerns are demonstrated in design, not this probe.

4. Latency claim discipline

Retrieval latency (min-of-5; see `baseline_protocol.md §3`): p50 ≈ 0.74 ms, p95 ≈ 0.84 ms — for an in-memory list. This is **not** a production latency claim; the 150 ms invariant is tested against the real store in D6. Documented here so no one reads the small number as evidence for production performance.

5. Productive value of each failure

Failure	Doesn't it just fail?	What it buys the project
F1 staleness wins	Stale fact injected → wrong action	Sharp, quantitative case for supersession + dating + explicit conflict signal (D4 R4/R6)
F2 sensitive leak	Total leak	Concrete repro for the PII guardrail invariant #5 and admission policy; D6 acceptance test = "same query after guardrail, leak = 0.0"
F3 false positive	Spurious match	Exact failure this crosses the "I don't know" fallback and QA threshold; cold-start contract for UX
F5 cross-user	Almost-collision	Prime justification for database-level tenant isolation (not just app-layer) — feeds the D6 tenant-isolation test gate
F6 metrics illusion	Single-metric reporting hides the failure pattern	Forces the R8 harness to measure 5 dimensions, not just recall

None of these failures required a real user to reproduce; all are captured in 10 fixed cases — the baseline is a **reproducible accusation** against every single-signal, store-everything design.

6. Explicit non-claims

- NOT claimed: a neural/embedding design would pass these cases (it would need the same R1–R6 structure; TF-IDF is the *fair/harsher* stand-in — protocol §3).
- NOT claimed: D3 measures system latency, cross-tenant adversarial leakage, or generation quality. Those are D6.
- NOT claimed: admission policies are easy to tune without cost; [A: admission accuracy] remains open and becomes a D4 design measurement.

7. Conclusion in one sentence

The naive baseline gives the project its falsification backbone: **a retriever alone recalls everything (recall@5 = 1.0) and still fails to act correctly (task-level 0.67), leaks all sensitive content it holds (injection 1.0), and fabricates matches when none exist (0.50)**, and each of these measured failures maps to an exact capability the final design must own.

End of report. Data: `baseline_results.csv`, `error_examples.jsonl`, `summary.json`; method: `baseline_protocol.md`. Jumper for D1 tags is `assumption_to_result.md`.